

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

А. В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Создание таблиц базы данных

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Получение навыков создания и организации реляционной базы данных при помощи PostgreSQL.

2 Задание и исходные данные

Вариант 5.

Создайте базу данных для хранения следующих сведений: ВУЗ, студент, группа, факультет, конференция, тема доклада, программа конференции.

3 Ход выполнения задания

На рисунке 1 представлена схема базы данных.

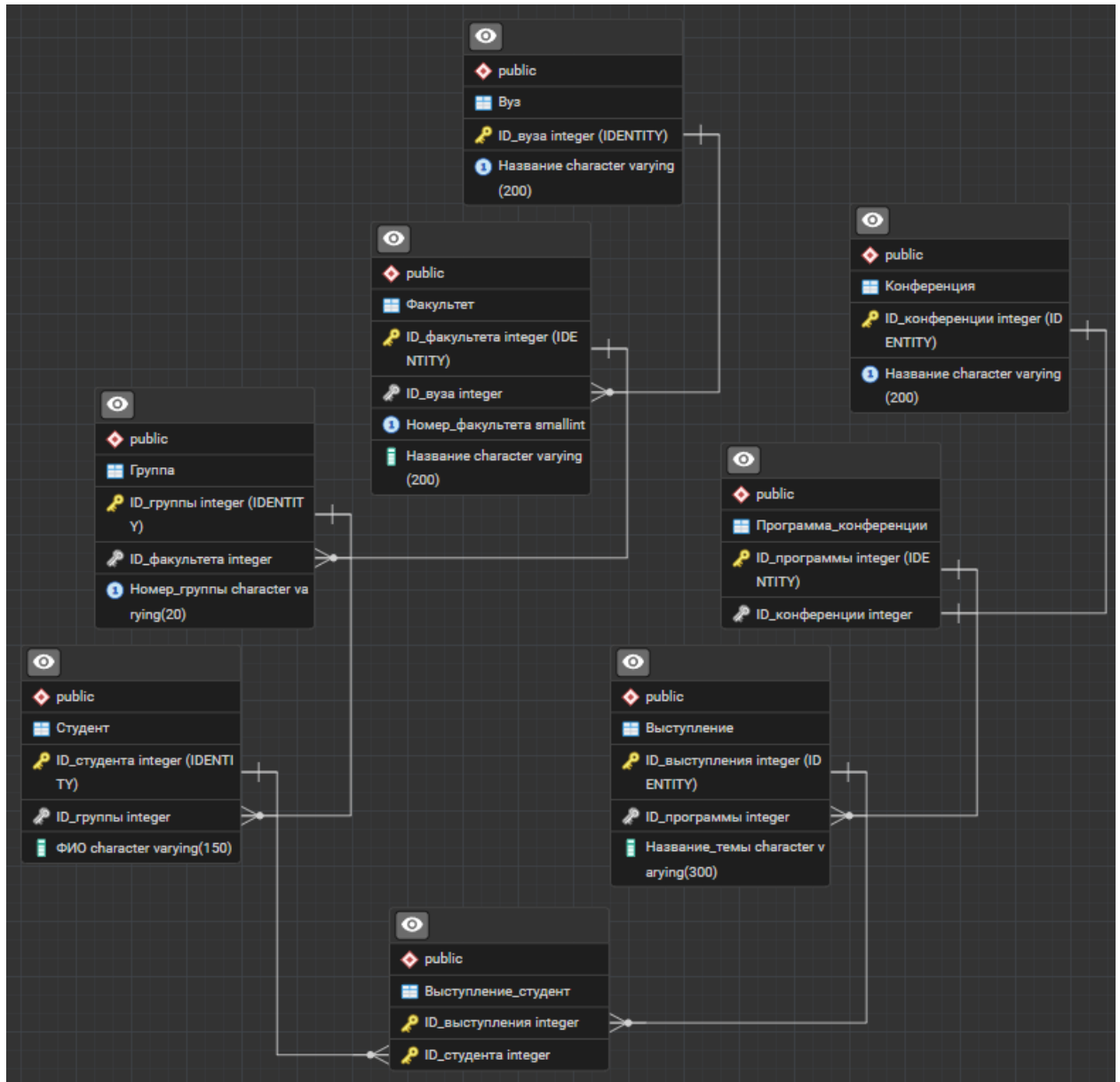


Рисунок 1 – Схема базы данных

Для первичных ключей во всех таблицах, кроме связующей, использован тип integer с конструкцией generated by default as identity primary key, что по смыслу совпадает с int identity в примерах Microsoft SQL Server из методички. Внешние ключи имеют тип integer и оформлены отдельными ограничениями constraint с именами на латинице в стиле fk_имя: fk_fakultet_vuz, fk_grupa_fakultet, fk_student_grupa, fk_programma_konferenciya,

fk_vystuplenie_programma, fk_vs_vystuplenie, fk_vs_student; для каждого указаны on delete cascade и on update cascade. Расширение pgcrypto не используется, так как идентификаторы не uuid.

Таблица Вуз содержит список вузов. Поле Название varchar обязательно, уникально и дополнительно ограничено проверкой check на непустую строку после функции trim.

Таблица Факультет хранит факультеты вуза. Поле Номер_факультета smallint обязательно и больше нуля по check. Пара ID_вуза и Номер_факультета уникальна в пределах вуза. Связь с Вуз задаётся ограничением fk_fakultet_vuz.

Таблица Группа описывает учебные группы на факультете. Номер_группы varchar обязателен с проверкой на непустоту после trim. Уникальность пары ID_факультета и Номер_группы. Внешний ключ на Факультет - fk_grupa_fakultet.

Таблица Студент содержит ФИО varchar и ссылку на одну группу через fk_student_grupa. ФИО проверяется на непустоту после trim.

Таблица Конференция хранит названия конференций varchar с уникальностью и проверкой на непустое название после trim.

Таблица Программа_конференции связывает каждую конференцию не более чем с одной программой за счёт ограничения unique на ID_конференции. Ссылка на Конференция оформлена как fk_programma_konferenciya.

Таблица Выступление задаёт тему доклада varchar Название_темы в рамках одной программы; тема доклада выделена полем без отдельной таблицы справочника тем. Связь с Программа_конференции - fk_vystuplenie_programma.

Таблица Выступление_студент реализует связь многие-ко-многим между выступлением и студентом. Составной первичный ключ из двух целочисленных столбцов. Внешние ключи fk_vs_vystuplenie и fk_vs_student с теми же правилами каскада, что и у остальных связей.

4 Вывод

В ходе работы спроектирована и создана в PostgreSQL база данных vuz_lr1 для варианта 5, обеспечивающая хранение сведений о вузе, факультетах, группах и студентах, а также о конференциях, программах, выступлениях и участии студентов. Заданы первичные и внешние ключи, уникальность и ограничения check, при удалении и обновлении родительских записей для внешних ключей использованы правила cascade где это соответствует иерархии данны.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
DROP TABLE IF EXISTS "Выступление_студент" CASCADE;
DROP TABLE IF EXISTS "Выступление" CASCADE;
DROP TABLE IF EXISTS "Программа_конференции" CASCADE;
DROP TABLE IF EXISTS "Конференция" CASCADE;
DROP TABLE IF EXISTS "Студент" CASCADE;
DROP TABLE IF EXISTS "Группа" CASCADE;
DROP TABLE IF EXISTS "Факультет" CASCADE;
DROP TABLE IF EXISTS "Вуз" CASCADE;

CREATE TABLE "Вуз" (
    "ID_вуза" INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY
    KEY,
    "Название" VARCHAR(200) NOT NULL UNIQUE CHECK
    (char_length(trim(both FROM "Название"))) > 0)
);

CREATE TABLE "Факультет" (
    "ID_факультета" INTEGER GENERATED BY DEFAULT AS IDENTITY
    PRIMARY KEY,
    "ID_вуза" INTEGER NOT NULL,
    "Номер_факультета" SMALLINT NOT NULL CHECK ("Номер_факультета"
    > 0),
    "Название" VARCHAR(200),
    UNIQUE ("ID_вуза", "Номер_факультета"),

    -- Факультет принадлежит одному вузу
```

```
CONSTRAINT fk_fakultet_vuz
    FOREIGN KEY ("ID_вуза")
    REFERENCES "Вуз" ("ID_вуза")
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

```
CREATE TABLE "Группа" (
    "ID_группы" INTEGER GENERATED BY DEFAULT AS IDENTITY
    PRIMARY KEY,
    "ID_факультета" INTEGER NOT NULL,
    "Номер_группы" VARCHAR(20) NOT NULL CHECK (char_length(trim(both
    FROM "Номер_группы"))) > 0),
    UNIQUE ("ID_факультета", "Номер_группы"),
```

```
-- Группа закреплена за факультетом
CONSTRAINT fk_grupa_fakultet
    FOREIGN KEY ("ID_факультета")
    REFERENCES "Факультет" ("ID_факультета")
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

```
CREATE TABLE "Студент" (
    "ID_студента" INTEGER GENERATED BY DEFAULT AS IDENTITY
    PRIMARY KEY,
    "ID_группы" INTEGER NOT NULL,
    "ФИО" VARCHAR(150) NOT NULL CHECK (char_length(trim(both FROM
    "ФИО"))) > 0),
```



```

-- Студент учится в одной группе
CONSTRAINT fk_student_grupa
    FOREIGN KEY ("ID_группы")
    REFERENCES "Группа" ("ID_группы")
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

CREATE TABLE "Конференция" (
    "ID_конференции" INTEGER GENERATED BY DEFAULT AS IDENTITY
    PRIMARY KEY,
    "Название" VARCHAR(200) NOT NULL UNIQUE CHECK
(char_length(trim(both FROM "Название"))) > 0
);

CREATE TABLE "Программа_конференции" (
    "ID_программы" INTEGER GENERATED BY DEFAULT AS IDENTITY
    PRIMARY KEY,
    "ID_конференции" INTEGER NOT NULL UNIQUE,

-- Одна программа на одну конференцию
CONSTRAINT fk_programma_konferenciya
    FOREIGN KEY ("ID_конференции")
    REFERENCES "Конференция" ("ID_конференции")
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

CREATE TABLE "Выступление" (

```

"ID_выступления" INTEGER GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY,

"ID_программы" INTEGER NOT NULL,

"Название_темы" VARCHAR(300) NOT NULL CHECK (char_length(trim(both
FROM "Название_темы"))) > 0),

-- Доклад входит в программу конференции

CONSTRAINT fk_vystuplenie_programma

FOREIGN KEY ("ID_программы")

REFERENCES "Программа_конференции" ("ID_программы")

ON DELETE CASCADE

ON UPDATE CASCADE

);

CREATE TABLE "Выступление_студент" (

"ID_выступления" INTEGER NOT NULL,

"ID_студента" INTEGER NOT NULL,

PRIMARY KEY ("ID_выступления", "ID_студента"),

-- Участие в конкретном выступлении

CONSTRAINT fk_vs_vystuplenie

FOREIGN KEY ("ID_выступления")

REFERENCES "Выступление" ("ID_выступления")

ON DELETE CASCADE

ON UPDATE CASCADE,

CONSTRAINT fk_vs_student

FOREIGN KEY ("ID_студента")

REFERENCES "Студент" ("ID_студента")

ON DELETE CASCADE

```
ON UPDATE CASCADE  
);
```